AFRICAN INSTITUTE FOR MATHEMATICAL SCIENCES

(AIMS RWANDA, KIGALI)

Name: Darix SAMANI SIEWE Assignment Number: 1

Course: Introduction to Neural Network Date: March 21, 2026

Least squares minimization problem

Suppose that we are given a set of observations $\{(x_1, y_1), \ldots, (x_n, y_n)\}$ where $x_i \in \mathbb{R}^p$ and $y_i \in \mathbb{R}$ for all i. A regression problem consists of finding a function f such that, for every i, $f(x_i)$ is as close as possible to y_i in terms of mean squared error.
In other words, we want to find a function f that solves the minimization problem:

$$\min_{f \in F} \sum_{i=1}^{n} (y_i - f(x_i))^2$$

where F is a family of functions. If F is the family of linear functions, the problem becomes finding a constant w_0 and a weight vector $w \in \mathbb{R}^p$ that solve :

$$\min_{w_0 \in \mathbb{R}, w \in \mathbb{R}^p} \sum_{i=1}^{n} \left(y_i - w_0 - x_i^T w\right)^2 \tag{1}$$

where x^T means "transpose of x" and

$$x_i^T w = \sum_{i=1}^{n} x_{ij} w_j$$

Part A : Real quadratic function

Consider a simplified case with one observation, $x_1 = -2$ and $y_1 = 1$.
3. Rewrite problem (1) for this case, assuming the parameter $w_0 = 0$.
Setting $w_0 = 0$ and substituting the single observation into (1) gives

$$\min_{w \in \mathbb{R}} \mathcal{L}(w), \qquad \mathcal{L}(w) \;=\; \left(y_1 - x_1 w\right)^2 \;=\; \left(1 - (-2)w\right)^2 \;=\; (1 + 2w)^2.$$

4. Find the exact solution.

The loss is a non-negative quadratic, so its unique minimiser is found by setting its derivative to zero.

$$\mathcal{L}'(w) = 2(1+2w)\cdot 2 = 4(1+2w).$$

$$\mathcal{L}'(w) = 0 \implies 1+2w = 0 \implies \boxed{w^* = -\tfrac{1}{2}}.$$

The minimum value is $\mathcal{L}(w^*) = 0$.
5. Write the gradient descent algorithm for this problem.
Since $\mathcal{L}'(w) = 4(1+2w)$, the gradient descent update rule is

$$w^{(t+1)} = w^{(t)} - \alpha\,\mathcal{L}'\big(w^{(t)}\big) = w^{(t)} - 4\alpha\big(1+2w^{(t)}\big)\,.$$

Algorithm 1 Gradient Descent for $\mathcal{L}(w) = (1+2w)^2$

Input: initial point $w^{(0)}$, learning rate α, tolerance ε
$t \leftarrow 0$
while $\big|\mathcal{L}'\big(w^{(t)}\big)\big| \geq \varepsilon$ **do**
  $w^{(t+1)} \leftarrow w^{(t)} - \alpha \cdot \mathcal{L}'\big(w^{(t)}\big)$
  $t \leftarrow t+1$
end while
return $w^{(t)}$

6. Apply the algorithm starting from $w_0 = 0$ with learning rate $\alpha = 0.1$.
How many iterations are needed to converge if the stopping criterion is : $|L'(w)| < 0.1$?

Iteration t	$w^{(t)}$	$\mathcal{L}'\big(w^{(t)}\big)$	$\big|\mathcal{L}'\big(w^{(t)}\big)\big| < 0.1$?
0	0	$4(1+0) = 4$	No
1	-0.4	$4(1-0.8) = 0.8$	No
2	-0.48	$4(1-0.96) = 0.16$	No
3	-0.496	$4(1-0.992) = 0.032$	**Yes**

The recurrence is $w^{(t+1)} = 0.2\,w^{(t)} - 0.4$, with error shrinking by factor 0.2 each step. **Three iterations** are required to satisfy $|\mathcal{L}'(w)| < 0.1$.
7. Compute 4 iterations starting from $w_0 = 0$ with $\alpha = 1$. What do you notice ?
The update becomes $w^{(t+1)} = -7w^{(t)} - 4$ (the eigenvalue of the map is -7, with $|-7| > 1$).

Iteration t	$w^{(t)}$	$\mathcal{L}'\big(w^{(t)}\big)$	$\big|\mathcal{L}'\big(w^{(t)}\big)\big| < 1$?
0	0	4(1+0) = 4	No
1	-4	4(1-8)=-28	No
2	24	4(1-48)= -188	No
3	-172	4(1-344)= -1372	No
4	1,200	4(1-2400)=-9596	No

Observation: The iterates diverge. A learning rate of $\alpha = 1$ is far too large; the update overshoots the minimum at every step and the loss grows without bound.
8. Compute 4 iterations starting from $w_0 = 0$ with $\alpha = 0.001$. What do you notice ?

The update rule with $\alpha = 0.001$ is

$$w^{(t+1)} = w^{(t)} - 0.001 \cdot 4\big(1 + 2w^{(t)}\big) = 0.992\, w^{(t)} - 0.004.$$

The contraction factor is $0.992 < 1$, so the sequence converges in principle, but extremely slowly.

Iteration t	$w^{(t)}$	$\mathcal{L}'\big(w^{(t)}\big) = 4\big(1 + 2w^{(t)}\big)$	$\big\lvert\mathcal{L}'\big(w^{(t)}\big)\big\rvert < 0.1$?
0	0	4.000	No
1	$-0.004\,000$	3.968	No
2	$-0.007\,968$	3.936	No
3	$-0.011\,904$	3.905	No
4	$-0.015\,809$	3.874	No

Observation: The gradient decreases by less than 0.03 per iteration and remains close to its initial value of 4. After 4 steps, $w^{(4)} \approx -0.016$, which is still very far from the exact solution $w^* = -0.5$. A rough estimate shows that roughly $\frac{4}{0.032} \approx 125$ iterations would be needed just to bring $|\mathcal{L}'(w)|$ below 0.1, and thousands would be needed to reach near-optimal accuracy. The step size $\alpha = 0.001$ is far too small: it satisfies the stability condition $\alpha < \frac{1}{L} = \frac{1}{4}$ comfortably, but wastes computational effort.

9. Using a constant step size is not always a good idea because choosing the step size is difficult. Instead, we can adapt the learning rate at each iteration using backtracking line search (Armijo rule).

Backtracking line search:

Choose constants: $0 \le a \le \frac{1}{2}$, $0 \le b \le 1$

$\tau \leftarrow \alpha$
while $\mathcal{L}(w - \tau\nabla\mathcal{L}(w)) > \mathcal{L}(w) - a\tau\|\nabla\mathcal{L}(w)\|_2^2$ **do**
  $\tau \leftarrow b \cdot \tau$
end while

---

where $\nabla\mathcal{L}(w)$ is the gradient and $\|\nabla\mathcal{L}(w)\|_2^2 = \nabla\mathcal{L}(w)^\top\nabla\mathcal{L}(w)$.
Run gradient descent with backtracking starting from $w_0 = 0$, using $\alpha = 1$ and $a = b = \frac{1}{2}$.

a) Write the gradient descent algorithm with backtracking.

Algorithm 2 Gradient Descent with Backtracking (Armijo Rule)

Input: $w^{(0)}$, $\alpha > 0$, $0 \le a \le \frac{1}{2}$, $0 \le b \le 1$, tolerance ε
$t \leftarrow 0$
while $\big|\mathcal{L}'\big(w^{(t)}\big)\big| \ge \varepsilon$ **do**
  $g \leftarrow \mathcal{L}'\big(w^{(t)}\big)$
  $\tau \leftarrow \alpha$
  while $\mathcal{L}\big(w^{(t)} - \tau g\big) > \mathcal{L}\big(w^{(t)}\big) - a\,\tau\,g^2$ **do**
    $\tau \leftarrow b \cdot \tau$
  end while
  $w^{(t+1)} \leftarrow w^{(t)} - \tau g$
  $t \leftarrow t + 1$
end while
return $w^{(t)}$

---

b) Run gradient descent with backtracking starting from $w_0 = 0$, using $\alpha = 1$ and$a = b = \frac{1}{2}$

Iteration $t = 0$. We have $w^{(0)} = 0$, $\mathcal{L}(0) = 1$, $g = \mathcal{L}'(0) = 4$, and $g^2 = 16$. The Armijo condition to satisfy is

$$\mathcal{L}(0 - \tau \cdot 4) \;\leq\; 1 - \tfrac{1}{2}\tau \cdot 16 \;=\; 1 - 8\tau.$$

τ	$w_{\text{new}} = -4\tau$	$\mathcal{L}(w_{\text{new}}) = (1 + 2(-4\tau))^2$	$1 - 8\tau$ (RHS)	
1	-4	$(1-8)^2 = 49$	-7	fail
$\frac{1}{2}$	-2	$(1-4)^2 = 9$	-3	fail
$\frac{1}{4}$	-1	$(1-2)^2 = 1$	-1	fail
$\frac{1}{8}$	$-\frac{1}{2}$	$(1-1)^2 = 0$	0	**pass**

The condition is met (with equality) at $\tau^* = \frac{1}{8}$.

$$w^{(1)} = 0 - \tfrac{1}{8} \cdot 4 = -\tfrac{1}{2} = w^*.$$

Since $\mathcal{L}'(w^{(1)}) = 4(1 + 2(-\frac{1}{2})) = 0 < 0.1$, the algorithm stops immediately. **One gradient descent step suffices.**

c) Count how many times you evaluate the function L and its derivative L'. Then compare this with standard gradient descent (without backtracking), starting from $w_0 = 0$ with $\alpha = 0.1$.

	Evaluations of $\mathcal{L}$	Evaluations of $\mathcal{L}'$
Backtracking GD ($\alpha = 1$, $a = b = \frac{1}{2}$)	5	2
Standard GD ($\alpha = 0.1$)	0	4

Detail for backtracking: one evaluation of $\mathcal{L}(w^{(0)})$ before the inner loop, four evaluations of $\mathcal{L}$ inside the backtracking loop ($\tau = 1, \frac{1}{2}, \frac{1}{4}, \frac{1}{8}$), and two evaluations of $\mathcal{L}'$ (once to obtain the gradient at $w^{(0)}$ and once to check the stopping criterion at $w^{(1)}$).

Detail for standard GD ($\alpha = 0.1$): $\mathcal{L}'$ is evaluated once per iteration (it drives both the update and the stopping check), for a total of 4 evaluations over 3 iterations plus the final check. No evaluations of $\mathcal{L}$ are needed.

Conclusion: Backtracking uses more evaluations of $\mathcal{L}$ per gradient step, but it reaches the solution in a single step regardless of the initial learning rate. Standard gradient descent with a well-tuned $\alpha = 0.1$ uses fewer total evaluations here, but choosing the right fixed step size requires prior knowledge of the problem. In higher-dimensional or ill-conditioned problems, backtracking reliably avoids divergence and slow convergence without manual tuning.

Part B : Quadratic function in $\mathbb{R}^{p+1}$

Recall that in linear regression, the minimization problem used to estimate the coefficients (or weights) is:

$$\min_{(\beta_0,\beta_1)\in\mathbb{R}^2} \frac{1}{2}\sum_{i=1}^{n}(y_i - \beta_0 - \beta_1 x_i)^2$$

11. Check that if we define

$$X = \begin{pmatrix} 1 & x_{11} \\ \vdots & \vdots \\ 1 & x_{n1} \end{pmatrix}, \quad y = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix}, \quad w = \begin{bmatrix} \beta_0 \\ \beta_1 \end{bmatrix}$$

then,

$$\frac{1}{2}\sum_{i=1}^{n}(y_i - \beta_0 - \beta_1 x_i)^2 = \frac{1}{2}(y - Xw)^\top (y - Xw) = \frac{1}{2}\|y - Xw\|_2^2 = \mathcal{L}(w)$$

Define

$$X = \begin{pmatrix} 1 & x_{11} \\ \vdots & \vdots \\ 1 & x_{n1} \end{pmatrix} \in \mathbb{R}^{n\times 2}, \qquad y = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix} \in \mathbb{R}^n, \qquad w = \begin{pmatrix} \beta_0 \\ \beta_1 \end{pmatrix} \in \mathbb{R}^2.$$

The i-th component of the product Xw is

$$(Xw)_i = 1 \cdot \beta_0 + x_{i1} \cdot \beta_1 = \beta_0 + \beta_1 x_i,$$

so the i-th component of the residual vector $y - Xw$ is $y_i - \beta_0 - \beta_1 x_i$. Therefore

$$(y - Xw)^\top (y - Xw) = \sum_{i=1}^{n}\big(y_i - \beta_0 - \beta_1 x_i\big)^2,$$

and, dividing by two and using the standard notation $\|\cdot\|_2^2 = (\cdot)^\top(\cdot)$,

$$\boxed{\mathcal{L}(w) \;=\; \frac{1}{2}\sum_{i=1}^{n}(y_i - \beta_0 - \beta_1 x_i)^2 \;=\; \frac{1}{2}(y - Xw)^\top (y - Xw) \;=\; \frac{1}{2}\|y - Xw\|_2^2.}$$

12) Compute the gradient of $\mathcal{L}$. You can check that:

$$\nabla\mathcal{L}(w) = X^\top (y - Xw).$$

Expanding the quadratic form gives

$$\mathcal{L}(w) = \frac{1}{2}\Big(y^\top y \;-\; 2w^\top X^\top y \;+\; w^\top X^\top X w\Big).$$

Using standard matrix-calculus identities ($\nabla_w(a^\top w) = a$ and $\nabla_w(w^\top A w) = 2Aw$ for symmetric A), we differentiate term by term:

$$\begin{aligned} \nabla_w\big[\tfrac{1}{2}y^\top y\big] &= \mathbf{0}, \\ \nabla_w\big[-w^\top X^\top y\big] &= -X^\top y, \\ \nabla_w\big[\tfrac{1}{2}w^\top X^\top X w\big] &= X^\top X w. \end{aligned}$$

Summing these terms yields

$$\nabla\mathcal{L}(w) = X^\top Xw - X^\top y = X^\top(Xw - y) = -X^\top(y - Xw).$$

Note on sign convention. The formula given in the problem statement, $\nabla\mathcal{L}(w) = X^\top(y - Xw)$, uses a sign convention in which the gradient points *toward* the residual; both forms are equivalent up to sign and lead to the same descent direction when the update reads $w \leftarrow w - \alpha\nabla\mathcal{L}(w)$. Throughout the remainder we adopt the standard convention $\nabla\mathcal{L}(w) = -X^\top(y - Xw) = X^\top Xw - X^\top y$, so that $\nabla\mathcal{L}(w^*) = \mathbf{0}$ at the ordinary-least-squares solution $w^* = (X^\top X)^{-1}X^\top y$.

13. Write the gradient descent algorithm in this case.

Starting from an initial weight vector $w^{(0)} \in \mathbb{R}^2$, each iteration is

$$w^{(t+1)} = w^{(t)} - \alpha\,\nabla\mathcal{L}\big(w^{(t)}\big) = w^{(t)} - \alpha\,X^\top\big(Xw^{(t)} - y\big)\,,$$

where $\alpha > 0$ is the (fixed) learning rate.

Algorithm 3 Gradient Descent for Linear Regression

Input: $w^{(0)} \in \mathbb{R}^2$, learning rate $\alpha > 0$, tolerance $\varepsilon > 0$, maximum iterations T
$t \leftarrow 0$
while $\|\nabla\mathcal{L}(w^{(t)})\|_2 \geq \varepsilon$ **and** $t < T$ **do**
  $g^{(t)} \leftarrow \nabla\mathcal{L}\big(w^{(t)}\big) = X^\top\big(Xw^{(t)} - y\big)$
  $w^{(t+1)} \leftarrow w^{(t)} - \alpha\,g^{(t)}$
  $t \leftarrow t + 1$
end while
return $w^{(t)}$

14. Describe the gradient descent algorithm with backtracking, and then implement it in Python. Write a function that performs this algorithm. The function should take as input: the starting point, the function to minimize, its gradient, the learning rate, the backtracking parameters, the convergence tolerance ε, and the maximum number of iterations.

```
def grad_desc_bt(w0, L, gL, alpha=0.1, a=0.5, b=0.5, eps=1e-4, maxiter=10)
```

Your answer should include:

- ⋆ a clear description (or pseudo-code) of the algorithm,
- ⋆ the corresponding Python implementation,
- ⋆ the returned values:
  - — the approximate solution,
  - — the value of the function at this point,
  - — the value of the gradient at this point,
  - — the number of iterations performed,
  - — and a boolean indicating whether the algorithm has converged.

Algorithm description

Rather than using a fixed step size, backtracking adapts the learning rate at each iteration via the **Armijo rule**: starting from a trial step size $\tau = \alpha$, we repeatedly shrink $\tau \leftarrow b\tau$ until the sufficient-decrease condition

$$\mathcal{L}\big(w^{(t)} - \tau\, g^{(t)}\big) \;\leq\; \mathcal{L}\big(w^{(t)}\big) \;-\; a\,\tau\,\big\|g^{(t)}\big\|_2^2$$

is satisfied, where $0 < a \leq \frac{1}{2}$ and $0 < b < 1$ are fixed constants.

Algorithm 4 Gradient Descent with Backtracking Line Search

Input: $w^{(0)} \in \mathbb{R}^{p+1}$, function $\mathcal{L}$, gradient $\nabla\mathcal{L}$, initial step $\alpha > 0$, Armijo constants $a, b \in (0,1)$, tolerance $\varepsilon > 0$, maximum iterations T
$t \leftarrow 0$
while $\|\nabla\mathcal{L}(w^{(t)})\|_2 \geq \varepsilon$ **and** $t < T$ **do**
  $g \leftarrow \nabla\mathcal{L}\big(w^{(t)}\big)$ ▷ compute gradient
  $\tau \leftarrow \alpha$ ▷ reset step size
  while $\mathcal{L}\big(w^{(t)} - \tau g\big) > \mathcal{L}\big(w^{(t)}\big) - a\,\tau\,\|g\|_2^2$ **do**
    $\tau \leftarrow b \cdot \tau$ ▷ shrink step size
  end while
  $w^{(t+1)} \leftarrow w^{(t)} - \tau\, g$ ▷ gradient step with adapted rate
  $t \leftarrow t + 1$
end while
return $w^{(t)},\ \mathcal{L}(w^{(t)}),\ \nabla\mathcal{L}(w^{(t)}),\ t,\ \big(\|\nabla\mathcal{L}(w^{(t)})\|_2 < \varepsilon\big)$

Python implementation

Listing 1: Gradient descent with backtracking line search

```python
import numpy as np

def grad_desc_bt(
    w0,
    L,
    gL,
    alpha: float = 0.1,
    a: float     = 0.5,
    b: float     = 0.5,
    eps: float   = 1e-4,
    maxiter: int = 10,
):
    """
    Gradient descent with Armijo backtracking line search.

    Parameters
    ----------
    w0      : array-like, shape (p+1,)
              Initial weight vector.
    L       : callable  w -> float
              Objective function to minimise.
```

```
        gL      : callable  w -> ndarray
                  Gradient of L.
        alpha   : float, optional
                  Initial (maximum) step size.  Default 0.1.
        a       : float in (0, 0.5], optional
                  Armijo sufficient-decrease constant.  Default 0.5.
        b       : float in (0, 1),   optional
                  Step-size reduction factor.  Default 0.5.
        eps     : float, optional
                  Convergence tolerance on ||grad L(w)||_2.  Default 1e-4.
        maxiter : int, optional
                  Maximum number of outer iterations.  Default 10.

        Returns
        -------
        w         : ndarray, shape (p+1,)
                    Approximate minimiser.
        L_val     : float
                    Value of L at w.
        g_val     : ndarray, shape (p+1,)
                    Gradient of L at w.
        n_iter    : int
                    Number of outer iterations performed.
        converged: bool
                    True if ||g_val||_2 < eps on exit.
        """
        w = np.asarray(w0, dtype=float).copy()

        for n_iter in range(1, maxiter + 1):

            g = gL(w)                        # gradient at current point
            grad_norm = np.linalg.norm(g)

            # --- convergence check ------------------------------------
            if grad_norm < eps:
                break

            # --- backtracking (Armijo) inner loop -----------------------
            tau      = alpha
            L_cur    = L(w)
            suff_dec = a * grad_norm ** 2    # = a * ||g||^2  (scalar)

            while L(w - tau * g) > L_cur - tau * suff_dec:
                tau *= b
```

```python

            # -- gradient step with accepted step size ----------------------
            w = w - tau * g

        # final quantities
        g_val     = gL(w)
        L_val     = L(w)
        converged = bool(np.linalg.norm(g_val) < eps)

        return w, L_val, g_val, n_iter, converged
```

Return values

- `w` — the approximate minimiser $w^{(t)} \in \mathbb{R}^{p+1}$;
- `L_val` — the objective value $\mathcal{L}(w^{(t)})$;
- `g_val` — the gradient $\nabla\mathcal{L}(w^{(t)})$;
- `n_iter` — the number of outer gradient steps performed;
- `converged` — `True` if the stopping criterion $\|\nabla\mathcal{L}(w^{(t)})\|_2 < \varepsilon$ is met, `False` if the loop terminated due to `maxiter`.